

DT-2 — TECHNICAL API SPECIFICATION

1. Global Standards

- **Date Format:** All dates are strings in **ISO-8601** format: `YYYY-MM-DDTHH:mm:ssZ` (e.g., `2023-10-25T14:30:00Z`).
- **IDs:** All IDs are **UUID strings**.
- **Standard List Wrapper:** Every endpoint that returns a list (Transactions, Logs, Files) **must** use this structure:

```
{
  "data": [],
  "meta": {
    "total_count": 1250,
    "page": 1,
    "page_size": 50,
    "total_pages": 25
  }
}
```

2. Authentication & Identity

POST `/api/v1/auth/login`

- **Purpose:** The entry point.
- **Request Body:**

```
{ "email": "user@org.com", "password": "secure_password" }
```

- **Logic:**
 1. Verify password hash against `USERS` table.
 2. Join `USER_SITE_ACCESS` with `SITES` to see which locations this user can see.
 3. Generate a JWT containing `userId`, `tenantId`, and `role`.

- **Success Response (200):**

```
{
  "token": "eyJhbGciOi...",
  "user": { "id": "uuid", "email": "...", "role": "Viewer" },
  "sites": [
    { "site_id": "uuid-1", "site_name": "Mumbai Plant" },
    { "site_id": "uuid-2", "site_name": "Delhi Warehouse" }
  ]
}
```

3. Dashboard Endpoints (FR-1)

GET /api/v1/dashboard/summary

- **Logic:** Sum the columns in the `TRANSACTIONS` table for the currently selected `site_id`.
- **Calculation:**
 - `queued` = SUM(total_files) - SUM(uploaded + processing + failed + completed)
 - `failed` = SUM(failed_count)
 - ...and so on.
- **Response:**

```
{
  "counters": {
    "queued": 150,
    "in_progress": 45,
    "completed": 8500,
    "failed": 24
  },
  "last_updated": "2023-10-25T14:30:00Z"
}
```

GET /api/v1/dashboard/throughput?range=7d

- **Params:** `range` (7d, 30d, 24h).

- **Logic:** Group `FILES` by `completed_at` date and count.
- **Response:**

```
{
  "unit": "day",
  "series": [
    { "label": "2023-10-21", "value": 450 },
    { "label": "2023-10-22", "value": 510 }
  ]
}
```

4. Transaction Explorer (FR-2)

`GET /api/v1/transactions`

- **Params:**
 - `page` : int (default 1)
 - `pageSize` : int (default 20)
 - `status` : string (e.g., "Failed", "Processing")
 - `search` : string (Full or partial Transaction ID)
 - `from` : ISO Date
 - `to` : ISO Date
- **Logic:** Filters the `TRANSACTIONS` table. **Must** include `WHERE tenant_id = X`.
- **Response:**

```
{
  "data": [
    {
      "transaction_id": "TID-9921",
      "status": "Failed",
      "source": "S3_Bucket_Alpha",
      "file_stats": { "total": 100, "done": 95, "error": 5 },
      "times": { "started": "...", "updated": "..." }
    }
  ],
  "meta": { ... }
}
```

5. File Detail & Error Remediation (FR-2.5 & FR-3)

GET /api/v1/files/{fileId}/details

- **Logic:** Joins FILES + FILE_STEP_HISTORY + ERROR_CATALOG.
- **Response:**

```
{
  "file_info": {
    "id": "uuid",
    "name": "invoice_oct.pdf",
    "current_status": "Failed",
    "current_step": "Extraction"
  },
  "history": [
    {
      "step": "Upload",
      "status": "Success",
      "ts": "2023-10-21T10:00:00Z"
    },
    {
      "step": "Extraction",
      "status": "Failed",
      "ts": "2023-10-21T10:05:00Z",
      "error": {
        "code": "ERR_OCR_40",
        "message": "Text unreadable on page 2",
        "suggested_fix": "Please re-scan the document at 300DPI."
      }
    }
  ]
}
```

6. Error Analysis Page (FR-3)

GET /api/v1/errors/top-frequencies

- **Logic:**

1. Look at `FILE_STEP_HISTORY` where `status = 'Failed'`.
2. Group by `error_code`.
3. Order by Count Descending.
4. Limit 10.

- **Response:**

```
[
  { "code": "INVALID_DATE", "count": 450, "remediation": "Update date
format to YYYY-MM-DD" },
  { "code": "SCHEMA_MISMATCH", "count": 210, "remediation": "Verify
CSV column headers" }
]
```

7. Security & Business Rules (The "Secret Sauce")

Rule 1: The JWT Middleware

You don't write `tenantId` in the code of every function. You create a **Middleware** (a filter that runs before the API code).

1. It looks at the incoming Token.
2. It says: "Ah, this is Rahul. He belongs to **Tenant: Tata Steel**."
3. It attaches `tenantId = 'Tata-uuid'` to the request object.
4. Your SQL query becomes: `SELECT * FROM Files WHERE tenant_id = Request.tenantId`. **This prevents users from seeing other people's files.**

Rule 2: Input Validation

If the Frontend sends `pageSize=10000`, your API should reject it with a **400 Bad Request**.

- Max `pageSize` should be 100. This protects your database from crashing.

Rule 3: 404 vs 403

- If a user requests `/api/v1/files/123`:
 - If ID 123 **does not exist**: Return **404 Not Found**.
 - If ID 123 **exists but belongs to another company**: Return **403 Forbidden**. (Never tell them the file exists if they don't own it).
-